shown below, provides a simplified implementation of a message authentication scheme.

```
1.  import hashlib

2.  def hash_message(msg):
3.      if isinstance(msg, str):
4.          msg = msg.encode("utf-8")
5.      h = hashlib.sha256(msg).hexdigest()
6.      return h

7.  def create_mac(secret_key, message):
8.      combined = secret_key + message
9.      return hash_message(combined)

10. def verify_mac(secret_key, message, received_mac):
11.     expected_mac = create_mac(secret_key, message)
12.     return expected_mac == received_mac

13. secret_key = "my_shared_secret"
14. message = "Hello Bob"
15. print("Message:", message)

16. mac = create_mac(secret_key, message)
17. print("MAC:", mac)

18. ok = verify_mac(secret_key, message, mac)
19. print("Verification:", ok)

20. tampered = message + "!"
21. print("Tampered message:", tampered)
22. ok2 = verify_mac(secret_key, tampered, mac)
23. print("Verification on tampered message:", ok2)
```

Now, let us try to understand the working of the program. Line 1 imports Python's *hashlib* module so that SHA-256 hashing can be performed. Lines 2 to 6 define a function *hash_message( )* that computes the SHA-256 hash of a message, similar to the program 'digsign41.sage'. Line 3 checks if the message is a Python string. Line 4 converts the string to bytes (required by SHA-256). Line 5 computes the SHA-256 hash of the message stored in the variable *msg* and returns it as a hexadecimal string. Line 6 returns the SHA-256 hash of the message. Lines 7 to 9 define a function to create a Message Authentication Code (MAC). Line 8 concatenates the shared secret key with the message. Line 9 computes the SHA-256 hash of the concatenation, producing the MAC. Lines 10 to 12 define a function to verify the MAC of a message. Line 11 recomputes the MAC for the message using the shared secret key. Line 12 returns *True* if the recomputed MAC matches the received MAC, and returns *False* otherwise.

```
Message: Hello Bob
MAC: 88d52c24023d66398f418c7ffc894f4d5d3f6a88e524138c9944503f4c792724
Verification: True
Tampered message: Hello Bob!
Verification on tampered message: False
```

Figure 3: Output of a simple message authentication scheme using hashing

Line 13 defines the shared secret key that both the sender and receiver know, which in this case is "*my_shared_secret*". Line 14 defines the message to be authenticated, which in this case is "*Hello Bob*". Line 15 prints the message to the console. Line 16 creates the MAC for the message using the shared secret key. Line 17 prints the MAC to the console. Line 18 verifies the MAC for the original message, and the result is stored in the variable *ok*. Line 19 prints *True* if the message is authentic. Line 20 creates a tampered version of the message by appending an exclamation mark. Line 21 prints the tampered message. Line 22 attempts to verify the MAC for the tampered message, and the result is stored in the variable *ok2*. Line 23 will print *False*, as the tampered message fails verification.

Upon executing the program 'mac42.sage', the output shown in Figure 3 is obtained. The first line of the figure displays the message "*Hello Bob*" which is to be authenticated. The second line displays the MAC of the message, which in this case is *88d52c24023d66398f418c7ffc894f4d5d 3f6a88e524138c9944503f4c792724*. The third line indicates that the original message is authentic, and therefore the verification result is *True*. The figure also shows the modified message "*Hello Bob!*" (with an exclamation mark appended) fails verification, correctly producing the result *False*, since the message has been tampered with.

With this article, we are concluding our discussion of how SageMath can be used to implement various cybersecurity algorithms and schemes. In the next article in this series, we will start exploring SageMath libraries and packages that can be applied in the field of mathematics called number theory. The applications of number theory in computer science include fields like cybersecurity (examples of which we have already seen), algorithms, random number generation, error-correcting codes, computer graphics, machine learning, AI, blockchain, etc. Thus, implementing number-theoretic applications using SageMath is essential. END

By: Dr Deepu Benson

The author is a free software enthusiast, and his area of interest is theoretical computer science. He is an assistant professor (senior scale) at Manipal Institute of Technology, Bengaluru, and maintains a technical blog.